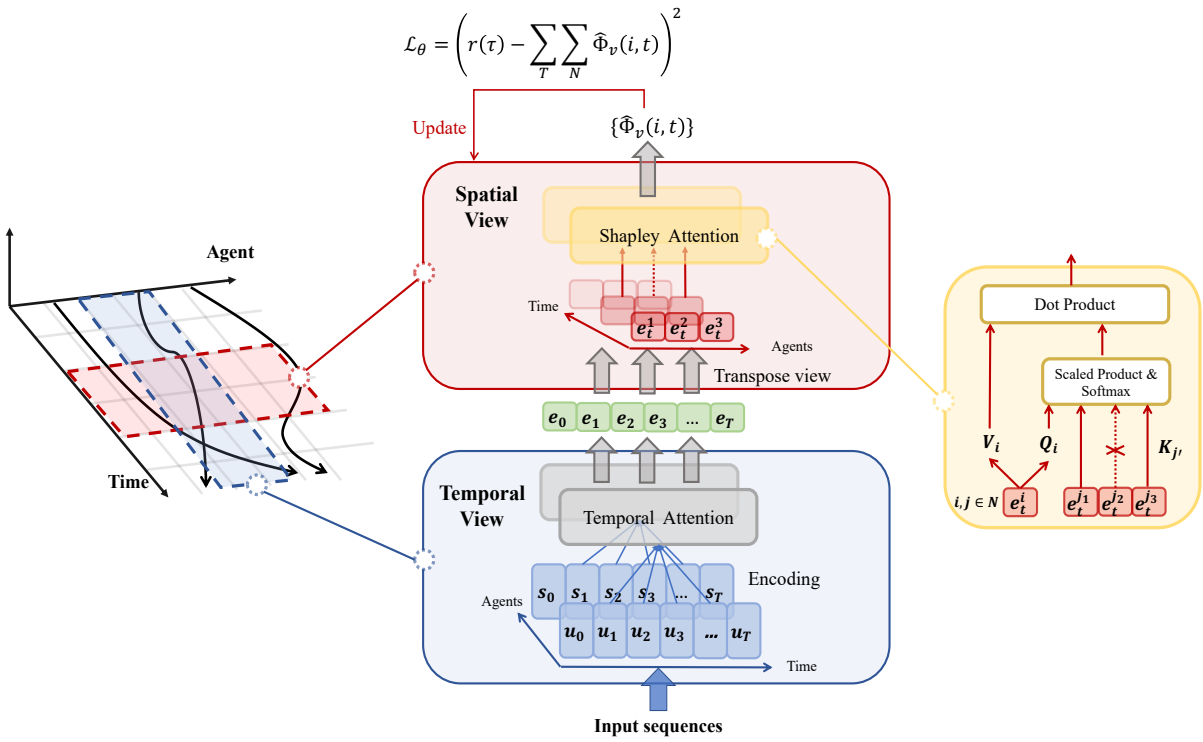

Figure 1: STAS framework. It contains a temporal attention module and a spatial Shapley attention module. Global states and actions of the entire episode are encoded and fed into the temporal attention module, which uses position embedding and time causality mask. The spatial Shapley attention module then approximates the Shapley value from previously learned representations in the spatial dimension. Finally, the model is updated based on the Shapley value approximation.

Value approximation:

$$\hat{\Phi}_v(i,t;\theta) = \frac{1}{K}\sum_{k=1}^{K} \hat{v}_i(C_k,t;\theta), \forall C_k \subseteq N\backslash\{i\}, \quad (10)$$

Although other estimation methods are available (Zhou et al. 2023), for the sake of simplicity and the absence of prior knowledge, we deem Monte Carlo estimation to be both necessary and sufficient for spatial reward decomposition.

Temporal Decomposition

Due to the efficiency property of the Shapley Value, the sum of all distributed payoffs equals the value of grand coalition. Therefore, we can get the estimated global reward at time t from the Shapley Value approximation of each agent, and the estimated final return can be written as:

$$\hat{r}_{\text{ep}}(\tau) = \sum_{t=0}^{T-1}\sum_{i=1}^{N} \hat{\Phi}_v(i,t;\theta). \quad (11)$$

In fact, the marginal contribution of an agent is seldom independent across time unless the relevant information from previous states has already been considered. To decompose the episodic return into a Markovian proxy reward function, the relations between actions and state transitions along trajectories must be established.

In practice, sequential modeling methods like Long short-term memory (LSTM) (Sak, Senior, and Beaufays 2014) or Transformer (Vaswani et al. 2017) are favored to encode the trajectory. Similar to Eq. (9), we adopt an attention function with causality masks on the state-action encoding $\boldsymbol{e}$ to incorporate temporal information. Specifically, we have

$$\begin{aligned} \boldsymbol{e}_t^i &= \text{MA}(\boldsymbol{q}_{i,t}^{\text{TEM}}, \boldsymbol{K}_i^{\text{TEM}}, \boldsymbol{V}_i^{\text{TEM}}) \\ &= \text{softmax}(\frac{\boldsymbol{q}_{i,t}^{\text{TEM}} \boldsymbol{K}_i^{\text{TEM}^{\text{T}}}}{\sqrt{d}} \odot \boldsymbol{M}^{\text{TEM}})\boldsymbol{V}_i^{\text{TEM}}, \end{aligned} \quad (12)$$

where the query matrix $\boldsymbol{q}_{i,t}^{\text{TEM}}$ is obtained by applying the trainable matrix $\boldsymbol{W}_Q^{\text{TEM}}$ to the output of a feed-forward layer f that encodes raw state and action embeddings. Similarly, the key and value matrices are generated by the trainable matrices $\boldsymbol{W}_K^{\text{TEM}}$ and $\boldsymbol{W}_V^{\text{TEM}}$, respectively. The self-attention weight is calculated using a softmax function applied to the scaled dot product of the query and key matrices, with the scaling factor $\sqrt{d}$ and the causality mask $\boldsymbol{M}^{\text{TEM}}$. $\boldsymbol{M}^{\text{TEM}}$ is a causality mask with its first t-th entries equal to 1, and remaining entries 0. It ensures that at each time step t, the model only attends to the states and actions before t.

Overall objective

With the estimated return from Eq. (11), we can train the return decomposition model by the following objective:

$$\mathcal{L}(\theta) = \mathop{\mathbb{E}}_{\tau\sim\mathcal{D}}\left[\left(r_{\text{ep}}(\tau) - \sum_{t=0}^{T-1}\sum_{i=1}^{N}\hat{\Phi}_v(i,t;\theta))\right)^2\right], \quad (13)$$

where $\hat{\Phi}_v(i,t;\theta)$ is the approximation of the Shapley Value of agent i at time t, $\mathcal{D}$ denotes the trajectories collected in the experience buffer and θ denotes the parameter of the return decomposition model. The details of the training algorithm are shown in algorithm 1.

Policy optimization For every policy, the complete trajectory's global states and actions can be fed into the return decomposition model to derive rewards for each agent at each time step, denoted as $\hat{\Phi}_v(i,t;\theta)$. Subsequently, the policy can be updated using algorithms like PPO (Schulman et al. 2017) or SAC (Haarnoja et al. 2018).

Algorithm 1: Spatial-Temporal Attention with Shapley value for Return Decomposition

Require: number of agents N; Initialized parameters θ of return decomposition model; Initialized parameters $\phi_1, \cdots, \phi_N$ of policy networks respectively; Experience buffer $B \leftarrow \emptyset$; Return decomposition model training frequency M

```
for k = 0, 1, ···, K do
    sample initial state s_0 ~ ρ_0
    for t = 0, 1, ···, T − 1 do
        sample actions u_t^i ~ π_{φ_i}(s_t^i), for i = 0, 1, ···, N
        Take joint actions u_t and observe next state s_{t+1}
    end for
    Collect episodic return r_ep(τ) and the trajectory
    τ = {s_0, u_0, ···, s_{T−1}, u_{T−1}}, store (τ, r_ep) into
    the buffer B
    Sample a batch of trajectories from buffer D ~ B
    Predict the decomposition reward {Φ̂_v(i, t; θ)} for
    each τ ∈ D
    Update parameter φ_i with independent trajectories
    τ_i = {(s_t^i, u_t^i, Φ̂_v(i, t; θ))}, for i = 0, 1, ···, N
    if k mod M is 0 then
        repeat
            Sample a batch of trajectories from buffer D' ~
            B and compute loss with Eq. (13)
            Update θ ← θ − ∇_θ L(θ)
        until Convergence
    end if
end for
```

Experiments

In this section, we conduct experiments to evaluate the performance and effectiveness of our proposed method on the challenging problem of spatial-temporal return decomposition. We evaluate our approach using two different implementations and test it on three different environments: a newly designed environment "Alice & Bob", as well as the *cooperative navigation* and *predator-prey* scenarios in the general multi-agent benchmark MPE (Lowe et al. 2017). The source code is available at `https://github.com/zowiezhang/STAS`.

Evaluate Algorithms

We compared our method with other several baseline algorithms in a scenario where rewards are given only at the game's end. Agents receive a 0 reward at each step and the sum of all rewards, i.e., the episodic return, after the game ends. We have evaluated two implementations of our method:

- **STAS (ours)** denotes the default implementation of our algorithm. First, we input the state-action sequence into a temporal attention module, which is designed to extract essential information from previous states. Next, we use a Shapley attention module to approximate the Shapley value for each agent. The predicted Shapley value is a measure of an agent's instant contribution to the team's final performance.
- **STAS-ML (ours)** is our alternative implementation, where "ML" stands for multi-layers. We organize temporal and spatial components into layers, each comprising a single temporal attention module and a single Shapley attention module. By stacking multiple layers, we construct our final prediction of the Shapley value. Each layer captures increasingly complex dependencies and interactions among agents, resulting in more accurate estimates of their individual contributions.

We compare with several existing baselines under the same settings, including **QMIX** (Rashid et al. 2020), **COMA** (Foerster et al. 2018) and **SQDDPG** (Wang et al. 2020a)

The specific implementation of our methods We utilize a temporal attention module to extract crucial information from previous states. To ensure causality, each state only receives information from earlier states through a causality mask. Positional embedding is employed to capture the relative positions of each state in the sequence. The Shapley attention module processes the learned embeddings for each agent to estimate their Shapley value. Our policy model is based on independent PPO (Schulman et al. 2017).

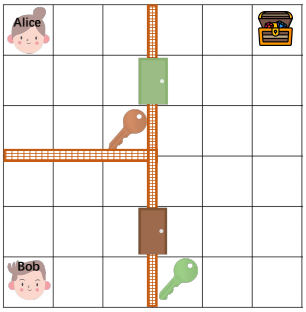

Figure 2: A simple demonstration of the designed extreme delayed reward environment Alice & Bob. When a key is reached, the corresponding door will open. To obtain the treasure, Alice must first retrieve the brown key to unlock the door to Bob's room. After Alice has unlocked the door, Bob can then retrieve the green key to open the door to Alice's room. With both doors unlocked, they can proceed to the treasure together.

Extreme delayed reward environment

In this experiment section, we designed a toy example environment called "Alice & Bob" to verify the effectiveness of

our method in an extremely delayed reward setting, as illustrated in Fig 2. In this task, agents Alice and Bob, unable to communicate, must collaboratively secure a treasure within a set number of steps, following a specific order and relying solely on their individual environmental states. They must adapt to the cooperative pattern dictated by the environment. The task's complexity is heightened by an extremely delayed reward system, where rewards accumulate and are only dispensed upon task completion, posing a substantial challenge to solving methods.

Environment settings At the beginning of the task, both Alice and Bob are locked in a room, but Alice can use the key in her room to unlock Bob's room. Once the key is touched, the door of Bob's room will be opened. Then Bob can get out of the room and find the key to Alice's room. After both of them get out of the room, they can go and get the treasure. The reward setting is that if any of them hit the wall when exploring, they will get a minus reward -0.2; if at the end of the game, they reach the treasure, they will get an extra reward of 200 and finish the task, and if they still not get the treasure within the maximum step, they will get the reward sum of each step as the whole return to force stop the task. Each agent has 4 different actions, namely, move_up, move_down, move_left and move_right.

Figure 3: Average agent rewards and reaching treasure rate with standard deviation for task Alice & Bob.

Results As presented in Fig 3, our methods STAS and STAS-ML outperform baselines, with STAS soaring to an average reward of 200 and a treasure rate of 0.9 at 30k episodes. QMIX improves slightly at 10k episodes but then declines. SQDDPG improves after 60k episodes, reaching an average reward of 130 and a treasure rate of 0.65. COMA is ineffective, with minimal rewards and a treasure rate of 0. The variance of QMIX and SQDDPG is high throughout training. Our results demonstrate our method's effectiveness in a challenging, extremely delayed reward environment, successfully learning cooperation patterns.

Multi-agent Particle Environment

Environment settings We selected two scenarios from MPE, namely *cooperative navigation* and *predator-prey*, to evaluate the performance of all algorithms. In task *cooperative navigation*, N agents are asked to reach L landmarks. Agents are rewarded based on how far any agent is from each landmark and penalized if they collide with other agents. In task *predator-prey*, N agents cooperate to catch M preys. Agents are rewarded when any of them catch the prey and penalized if they stay away from the preys. All preys are controlled by a pre-trained policy. In our settings, rewards are only revealed at the end of each episode.

Results The results in Fig 4 demonstrate our approach's superior performance and sample efficiency over all baselines, particularly in environments with delayed rewards. Baselines show increased variance and ineffectiveness as agent numbers rise, likely due to the complexity in assessing individual agent contributions in larger cooperative systems without mid-term global rewards. While STAS-ML surpasses STAS in the *predator-prey* task with three agents, it shows comparable results in other tasks. This implies that forgoing Shapley Value properties may reduce performance in complex scenarios but enhance sample efficiency in simpler, smaller-agent tasks.

Ablations

We conducted several ablation experiments to evaluate different components of the STAS. Fig 5(a) demonstrates the impact of different practical methods of the spatial transformer. It shows that when the sample number of Shapley value approximation is 1, not only is the performance the worst but the variance is also larger, making it less stable. When the sample number is 5, the performance is optimal, as increasing the sample number leads to a more accurate estimation of the true value. However, when the sample number is 3, the performance is already close to that of 5, indicating that the performance is good enough with 3 samples.

Fig 5(b) demonstrates the impact of using different numbers of layers in the STAS-ML. When the number of layers is 1, the performance shows higher variance and is less stable. This may be because a single layer is not sufficient to extract high-level hidden information. As the number of layers increases, the performance significantly improves. When the number of layers is 3 and 5, the performance is already very close, indicating that the model is able to extract the relevant information about the interactions between the agents with several layers.

Following Wang et al. (2020a), we conducted an experiment to verify the effectiveness of the Shapley value. We computed the Pearson coefficient correlating the credits assigned to each agent with the inverse of their distance to